

Mathematics for Machine Learning - Complete Cheat Sheet

1. LINEAR ALGEBRA

Vectors

Definition: Ordered array of numbers representing magnitude and direction

```
python

import numpy as np

# Vector creation
v = np.array([1, 2, 3])
w = np.array([4, 5, 6])

# Vector operations
addition = v + w          # [5, 7, 9]
scalar_mult = 3 * v       # [3, 6, 9]
dot_product = np.dot(v, w) # 1*4 + 2*5 + 3*6 = 32
magnitude = np.linalg.norm(v) #  $\sqrt{1^2 + 2^2 + 3^2} = \sqrt{14}$ 
```

Key Formulas:

- Dot Product: $\mathbf{v} \cdot \mathbf{w} = \sum(\mathbf{v}_i \times \mathbf{w}_i) = |\mathbf{v}||\mathbf{w}|\cos(\theta)$
- Magnitude: $\|\mathbf{v}\| = \sqrt{\sum \mathbf{v}_i^2}$
- Unit Vector: $\hat{\mathbf{v}} = \mathbf{v} / \|\mathbf{v}\|$

ML Use Cases: Feature vectors, embeddings, word2vec

Matrices

Definition: 2D array of numbers ($m \times n$)

```
python

# Matrix creation
A = np.array([[1, 2], [3, 4], [5, 6]]) # 3x2 matrix
B = np.array([[7, 8], [9, 10]])       # 2x2 matrix

# Matrix operations
transpose = A.T          # Swap rows/columns
matmul = A @ B           # Matrix multiplication
element_wise = A * 2     # Scalar multiplication
identity = np.eye(3)     # 3x3 identity matrix
```

Key Operations:

- **Matrix Multiplication:** $C = AB$ where $C[i,j] = \sum(A[i,k] \times B[k,j])$
 - Only valid when A is $(m \times n)$ and B is $(n \times p)$
- **Transpose:** A^T where $A^T[i,j] = A[j,i]$
- **Inverse:** A^{-1} where $AA^{-1} = I$ (only for square matrices)

```
python
```

```
# Matrix inverse and determinant
```

```
A_square = np.array([[1, 2], [3, 4]])
```

```
inverse = np.linalg.inv(A_square)
```

```
determinant = np.linalg.det(A_square) # ad - bc for 2x2
```

ML Use Cases: Neural network weights, data transformations, image processing

Eigenvalues & Eigenvectors

Definition: $Av = \lambda v$ (eigenvector v , eigenvalue λ)

```
python
```

```
A = np.array([[4, 2], [1, 3]])
```

```
eigenvalues, eigenvectors = np.linalg.eig(A)
```

```
# eigenvalues: scaling factors
```

```
# eigenvectors: directions that don't change under transformation
```

ML Use Cases: PCA (dimensionality reduction), PageRank, covariance matrices

Matrix Decompositions

Singular Value Decomposition (SVD):

```
python
```

```
# A = UΣV^T
```

```
A = np.array([[1, 2, 3], [4, 5, 6]])
```

```
U, S, Vt = np.linalg.svd(A)
```

```
# U: left singular vectors (m × m)
```

```
# S: singular values (diagonal)
```

```
# Vt: right singular vectors (n × n)
```

ML Use Cases: Recommender systems, image compression, LSA

2. CALCULUS

Derivatives

Definition: Rate of change of a function

Single Variable:

```
python
```

```
#  $f(x) = x^2$ 
```

```
#  $f'(x) = 2x$ 
```

```
from scipy.misc import derivative
```

```
f = lambda x: x**2
```

```
df = derivative(f, 3.0, dx=1e-6) #  $\approx 6$ 
```

Common Derivatives:

- $(x^n)' = n \cdot x^{n-1}$
- $(e^x)' = e^x$
- $(\ln x)' = 1/x$
- $(\sin x)' = \cos x$
- $(\cos x)' = -\sin x$

Chain Rule: $(f(g(x)))' = f'(g(x)) \cdot g'(x)$

```
python
```

```
# Example:  $f(x) = (2x + 1)^3$ 
```

```
#  $f'(x) = 3(2x + 1)^2 \cdot 2 = 6(2x + 1)^2$ 
```

Partial Derivatives

Definition: Derivative with respect to one variable, holding others constant

```
python
```

```
import sympy as sp

x, y = sp.symbols('x y')
f = x**2 + 3*x*y + y**2

#  $\partial f / \partial x$ 
df_dx = sp.diff(f, x) # 2x + 3y

#  $\partial f / \partial y$ 
df_dy = sp.diff(f, y) # 3x + 2y
```

ML Use Cases: Gradient descent, backpropagation

Gradient

Definition: Vector of all partial derivatives

$$\nabla f = [\partial f / \partial x_1, \partial f / \partial x_2, \dots, \partial f / \partial x_n]$$

```
python

def gradient(f, point):
    """Numerical gradient computation"""
    grad = np.zeros_like(point)
    h = 1e-5
    for i in range(len(point)):
        point_plus = point.copy()
        point_plus[i] += h
        point_minus = point.copy()
        point_minus[i] -= h
        grad[i] = (f(point_plus) - f(point_minus)) / (2*h)
    return grad

# Loss function
def loss(w):
    return np.sum(w**2)

w = np.array([3.0, 4.0])
grad = gradient(loss, w) # [6.0, 8.0]
```

Gradient Descent

Core Algorithm for ML Optimization:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \alpha \nabla L(\mathbf{w})$$

```
python
```

```
def gradient_descent(f, grad_f, initial_w, learning_rate=0.01, iterations=1000):
    w = initial_w.copy()
    history = [w.copy()]

    for i in range(iterations):
        grad = grad_f(w)
        w = w - learning_rate * grad
        history.append(w.copy())

    return w, history

# Example: Minimize  $f(x) = x^2 + y^2$ 
def f(w):
    return w[0]**2 + w[1]**2

def grad_f(w):
    return 2 * w

initial = np.array([10.0, 10.0])
optimal, history = gradient_descent(f, grad_f, initial)
```

ML Use Cases: Training neural networks, linear regression, logistic regression

3. PROBABILITY & STATISTICS

Probability Basics

Key Formulas:

- $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ (Union)
- $P(A \cap B) = P(A) \cdot P(B)$ (Independent events)
- $P(A|B) = P(A \cap B) / P(B)$ (Conditional probability)

Bayes' Theorem: $P(A|B) = P(B|A) \cdot P(A) / P(B)$

python

```
# Example: Email spam classification
```

```
#  $P(\text{Spam} | \text{"lottery"}) = P(\text{"lottery"} | \text{Spam}) \cdot P(\text{Spam}) / P(\text{"lottery"})$ 
```

```
p_spam = 0.3 # Prior
```

```
p_lottery_given_spam = 0.8
```

```
p_lottery_given_ham = 0.05
```

```
p_lottery = p_lottery_given_spam * p_spam + p_lottery_given_ham * (1 - p_spam)
```

```
p_spam_given_lottery = (p_lottery_given_spam * p_spam) / p_lottery
```

```
#  $\approx 0.77$ 
```

Random Variables & Distributions

Expected Value: $E[X] = \sum x_i \cdot P(x_i)$ (discrete) or $\int x \cdot f(x) dx$ (continuous)

Variance: $\text{Var}(X) = E[(X - \mu)^2] = E[X^2] - (E[X])^2$

Standard Deviation: $\sigma = \sqrt{\text{Var}(X)}$

```
python
```

```
from scipy import stats
```

```
# Normal/Gaussian Distribution
```

```
mu, sigma = 0, 1
```

```
x = np.random.normal(mu, sigma, 1000)
```

```
# PDF:  $f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \cdot e^{-(x-\mu)^2/(2\sigma^2)}$ 
```

```
pdf_value = stats.norm.pdf(0, mu, sigma)
```

```
# CDF
```

```
cdf_value = stats.norm.cdf(1.96, mu, sigma) #  $\approx 0.975$ 
```

Common Distributions:

```
python
```

```
# Bernoulli (coin flip)
bernoulli = stats.bernoulli.rvs(p=0.5, size=100)

# Binomial (n coin flips)
binomial = stats.binom.rvs(n=10, p=0.5, size=100)

# Uniform
uniform = stats.uniform.rvs(loc=0, scale=1, size=100)

# Exponential
exponential = stats.expon.rvs(scale=1, size=100)
```

Statistical Measures

```
python

data = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# Central tendency
mean = np.mean(data)      # 5.5
median = np.median(data)  # 5.5
mode = stats.mode(data, keepdims=True).mode[0]

# Spread
variance = np.var(data)    # Population variance
std = np.std(data)         # Population std
sample_var = np.var(data, ddof=1) # Sample variance (unbiased)

# Percentiles
q25 = np.percentile(data, 25) # 3.25
q75 = np.percentile(data, 75) # 7.75
```

ML Use Cases: Feature normalization, uncertainty quantification, Bayesian ML

4. INFORMATION THEORY

Entropy

Definition: Measure of uncertainty/information content

$$H(X) = -\sum P(x_i) \cdot \log_2(P(x_i))$$

```
python
```

```
def entropy(probabilities):
    """Shannon entropy"""
    probabilities = np.array(probabilities)
    probabilities = probabilities[probabilities > 0] # Avoid log(0)
    return -np.sum(probabilities * np.log2(probabilities))

# Fair coin: H = 1 bit
fair_coin = [0.5, 0.5]
print(entropy(fair_coin)) # 1.0

# Biased coin: H < 1 bit
biased_coin = [0.9, 0.1]
print(entropy(biased_coin)) # 0.469
```

Cross-Entropy

Definition: Measures difference between two probability distributions

$$H(p, q) = -\sum p(x) \cdot \log(q(x))$$

```
python

def cross_entropy(y_true, y_pred):
    """Cross-entropy loss"""
    epsilon = 1e-15 # Avoid log(0)
    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
    return -np.sum(y_true * np.log(y_pred))

# Classification example
y_true = np.array([1, 0, 0]) # One-hot encoded
y_pred = np.array([0.7, 0.2, 0.1]) # Model prediction
loss = cross_entropy(y_true, y_pred)
```

ML Use Cases: Classification loss function, neural network training

KL Divergence

Definition: Measures how one distribution differs from another

$$D_{KL}(P||Q) = \sum P(x) \cdot \log(P(x)/Q(x))$$

```
python
```

```
def kl_divergence(p, q):
    """KL divergence from Q to P"""
    p = np.array(p)
    q = np.array(q)
    return np.sum(p * np.log(p / q))

p = np.array([0.4, 0.6])
q = np.array([0.5, 0.5])
kl = kl_divergence(p, q) # Not symmetric: D_KL(P||Q) ≠ D_KL(Q||P)
```

ML Use Cases: Variational autoencoders, reinforcement learning

5. OPTIMIZATION

Convex Optimization

Definition: Finding minimum of convex function

Convex Function: $f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda)f(y)$ for $\lambda \in [0,1]$

```
python

# Example: Linear regression (convex)
def mse_loss(w, X, y):
    """Mean Squared Error (convex)"""
    predictions = X @ w
    return np.mean((predictions - y)**2)

# Closed-form solution:  $w^* = (X^T X)^{-1} X^T y$ 
def solve_linear_regression(X, y):
    return np.linalg.inv(X.T @ X) @ X.T @ y
```

Optimization Algorithms

Batch Gradient Descent:

```
python
```

```
def batch_gradient_descent(X, y, w, lr=0.01, epochs=100):
    m = len(y)
    for epoch in range(epochs):
        predictions = X @ w
        error = predictions - y
        gradient = (1/m) * X.T @ error
        w = w - lr * gradient
    return w
```

Stochastic Gradient Descent (SGD):

```
python

def sgd(X, y, w, lr=0.01, epochs=100):
    m = len(y)
    for epoch in range(epochs):
        for i in range(m):
            xi = X[i:i+1]
            yi = y[i:i+1]
            prediction = xi @ w
            error = prediction - yi
            gradient = xi.T @ error
            w = w - lr * gradient
    return w
```

Mini-batch Gradient Descent:

```
python

def mini_batch_gd(X, y, w, lr=0.01, batch_size=32, epochs=100):
    m = len(y)
    for epoch in range(epochs):
        indices = np.random.permutation(m)
        X_shuffled = X[indices]
        y_shuffled = y[indices]

        for i in range(0, m, batch_size):
            X_batch = X_shuffled[i:i+batch_size]
            y_batch = y_shuffled[i:i+batch_size]

            predictions = X_batch @ w
            error = predictions - y_batch
            gradient = (1/len(y_batch)) * X_batch.T @ error
            w = w - lr * gradient
    return w
```

Adam Optimizer (Adaptive Moment Estimation):

```
python

def adam_optimizer(grad_fn, w, lr=0.001, beta1=0.9, beta2=0.999, eps=1e-8, iterations=1000):
    m = np.zeros_like(w) # First moment
    v = np.zeros_like(w) # Second moment

    for t in range(1, iterations + 1):
        grad = grad_fn(w)

        m = beta1 * m + (1 - beta1) * grad
        v = beta2 * v + (1 - beta2) * (grad ** 2)

        m_hat = m / (1 - beta1 ** t) # Bias correction
        v_hat = v / (1 - beta2 ** t)

        w = w - lr * m_hat / (np.sqrt(v_hat) + eps)

    return w
```

6. REAL-WORLD ML APPLICATIONS

Linear Regression

```
python

from sklearn.linear_model import LinearRegression

#  $y = Xw + b$ 
# Minimize:  $L(w) = (1/2m) \sum (\hat{y}_i - y_i)^2$ 

X = np.array([[1], [2], [3], [4]])
y = np.array([2, 4, 6, 8])

model = LinearRegression()
model.fit(X, y)

# Manual implementation
X_bias = np.c_[np.ones(len(X)), X] # Add bias term
w = np.linalg.inv(X_bias.T @ X_bias) @ X_bias.T @ y
```

Logistic Regression

```
python

from sklearn.linear_model import LogisticRegression

# Sigmoid:  $\sigma(z) = 1 / (1 + e^{(-z)})$ 
# Loss:  $L = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$ 

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def logistic_loss(w, X, y):
    z = X @ w
    predictions = sigmoid(z)
    epsilon = 1e-15
    predictions = np.clip(predictions, epsilon, 1 - epsilon)
    return -np.mean(y * np.log(predictions) + (1 - y) * np.log(1 - predictions))
```

Neural Network Forward Pass

```
python

def relu(x):
    return np.maximum(0, x)

def softmax(x):
    exp_x = np.exp(x - np.max(x, axis=1, keepdims=True))
    return exp_x / np.sum(exp_x, axis=1, keepdims=True)

# Simple 2-layer network
def forward_pass(X, W1, b1, W2, b2):
    # Layer 1
    Z1 = X @ W1 + b1
    A1 = relu(Z1)

    # Layer 2 (output)
    Z2 = A1 @ W2 + b2
    A2 = softmax(Z2)

    return A2, (Z1, A1, Z2)
```

Backpropagation

```
python
```

```
def backward_pass(X, y, W1, b1, W2, b2, cache, lr=0.01):  
    Z1, A1, Z2 = cache  
    m = X.shape[0]  
  
    # Output layer gradient  
    dZ2 = A2 - y # Softmax + cross-entropy derivative  
    dW2 = (1/m) * A1.T @ dZ2  
    db2 = (1/m) * np.sum(dZ2, axis=0)  
  
    # Hidden layer gradient  
    dA1 = dZ2 @ W2.T  
    dZ1 = dA1 * (Z1 > 0) # ReLU derivative  
    dW1 = (1/m) * X.T @ dZ1  
    db1 = (1/m) * np.sum(dZ1, axis=0)  
  
    # Update weights  
    W1 -= lr * dW1  
    b1 -= lr * db1  
    W2 -= lr * dW2  
    b2 -= lr * db2  
  
    return W1, b1, W2, b2
```

Principal Component Analysis (PCA)

python

```
from sklearn.decomposition import PCA

# Dimensionality reduction via eigendecomposition

def manual_pca(X, n_components):
    # Center data
    X_centered = X - np.mean(X, axis=0)

    # Covariance matrix
    cov_matrix = np.cov(X_centered.T)

    # Eigendecomposition
    eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

    # Sort by eigenvalues
    idx = eigenvalues.argsort()[::-1]
    eigenvalues = eigenvalues[idx]
    eigenvectors = eigenvectors[:, idx]

    # Select top n components
    components = eigenvectors[:, :n_components]

    # Transform data
    X_transformed = X_centered @ components

    return X_transformed, components
```

7. KEY FORMULAS QUICK REFERENCE

Activation Functions

```
python
```

```

# Sigmoid:  $\sigma(x) = 1/(1 + e^{(-x)})$ 
sigmoid = lambda x: 1 / (1 + np.exp(-x))

# Tanh:  $\tanh(x) = (e^x - e^{(-x)})/(e^x + e^{(-x)})$ 
tanh = lambda x: np.tanh(x)

# ReLU:  $\max(0, x)$ 
relu = lambda x: np.maximum(0, x)

# Leaky ReLU:  $\max(ax, x)$ 
leaky_relu = lambda x, alpha=0.01: np.where(x > 0, x, alpha * x)

# Softmax:  $e^{x_i} / \sum e^{x_j}$ 
def softmax(x):
    exp_x = np.exp(x - np.max(x))
    return exp_x / np.sum(exp_x)

```

Loss Functions

```

python

# Mean Squared Error
mse = lambda y_true, y_pred: np.mean((y_true - y_pred)**2)

# Mean Absolute Error
mae = lambda y_true, y_pred: np.mean(np.abs(y_true - y_pred))

# Binary Cross-Entropy
def binary_crossentropy(y_true, y_pred):
    epsilon = 1e-15
    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
    return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

# Categorical Cross-Entropy
def categorical_crossentropy(y_true, y_pred):
    epsilon = 1e-15
    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
    return -np.sum(y_true * np.log(y_pred))

```

Regularization

```

python

```

```
# L1 Regularization (Lasso):  $\lambda \sum |w_i|$ 
```

```
l1_reg = lambda w, lambda_: lambda_ * np.sum(np.abs(w))
```

```
# L2 Regularization (Ridge):  $\lambda \sum w_i^2$ 
```

```
l2_reg = lambda w, lambda_: lambda_ * np.sum(w**2)
```

```
# Elastic Net:  $\lambda_1 \sum |w_i| + \lambda_2 \sum w_i^2$ 
```

```
elastic_net = lambda w, l1, l2: l1 * np.sum(np.abs(w)) + l2 * np.sum(w**2)
```

Distance Metrics

```
python
```

```
# Euclidean Distance:  $\sqrt{\sum (x_i - y_i)^2}$ 
```

```
euclidean = lambda x, y: np.sqrt(np.sum((x - y)**2))
```

```
# Manhattan Distance:  $\sum |x_i - y_i|$ 
```

```
manhattan = lambda x, y: np.sum(np.abs(x - y))
```

```
# Cosine Similarity:  $(x \cdot y) / (||x|| \cdot ||y||)$ 
```

```
cosine_sim = lambda x, y: np.dot(x, y) / (np.linalg.norm(x) * np.linalg.norm(y))
```

8. PRACTICAL TIPS

Feature Scaling

```
python
```

```
# Standardization (Z-score):  $(x - \mu) / \sigma$ 
```

```
def standardize(X):
```

```
    return (X - np.mean(X, axis=0)) / np.std(X, axis=0)
```

```
# Min-Max Normalization:  $(x - \min) / (\max - \min)$ 
```

```
def normalize(X):
```

```
    return (X - np.min(X, axis=0)) / (np.max(X, axis=0) - np.min(X, axis=0))
```

Train-Test Split

```
python
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Performance Metrics

```
python

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Classification
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred)
recall = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)

# Regression
from sklearn.metrics import mean_squared_error, r2_score
mse = mean_squared_error(y_true, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_true, y_pred)
```

Summary of When to Use What

Concept	When to Use	Example
Linear Algebra	Data representation, transformations	Image processing, PCA
Calculus	Optimization, backpropagation	Training neural networks
Probability	Uncertainty modeling	Bayesian inference, GANs
Information Theory	Loss functions	Classification tasks
Optimization	Model training	Gradient descent variants
Statistics	Data analysis, feature selection	EDA, hypothesis testing

9. ADVANCED LINEAR ALGEBRA CONCEPTS

Matrix Rank & Null Space

Rank: Number of linearly independent rows/columns

```
python

A = np.array([[1, 2, 3], [2, 4, 6], [3, 6, 9]])
rank = np.linalg.matrix_rank(A) # 1 (all rows are linearly dependent)

# Full rank matrix
B = np.array([[1, 2], [3, 4]])
rank_B = np.linalg.matrix_rank(B) # 2 (full rank)
```

ML Applications:

- Feature redundancy detection
- Determining if a system of equations has unique solution
- Dimensionality analysis in neural networks

Orthogonality & Projection

Orthogonal vectors: $v \cdot w = 0$

```
python
```

```

def is_orthogonal(v, w):
    return np.abs(np.dot(v, w)) < 1e-10

# Gram-Schmidt Orthogonalization
def gram_schmidt(vectors):
    """Convert vectors to orthonormal basis"""
    basis = []
    for v in vectors:
        # Subtract projections onto previous basis vectors
        w = v.copy()
        for b in basis:
            w -= np.dot(v, b) * b
        # Normalize
        if np.linalg.norm(w) > 1e-10:
            basis.append(w / np.linalg.norm(w))
    return np.array(basis)

# Projection of v onto w
def project(v, w):
    return (np.dot(v, w) / np.dot(w, w)) * w

```

ML Applications:

- QR decomposition in optimization
- Orthogonal regularization in neural networks
- Feature decorrelation

Trace & Frobenius Norm

```

python

A = np.array([[1, 2], [3, 4]])

# Trace: sum of diagonal elements
trace = np.trace(A) # 1 + 4 = 5

# Frobenius Norm: ||A||_F = sqrt(sum(a_ij^2))
frobenius = np.linalg.norm(A, 'fro') # sqrt(1^2 + 2^2 + 3^2 + 4^2) = sqrt(30)

# Properties:
# tr(A + B) = tr(A) + tr(B)
# tr(AB) = tr(BA)
# tr(A^T) = tr(A)

```

ML Applications:

- Nuclear norm regularization
- Matrix completion
- Similarity measures between matrices

Kronecker Product & Vectorization

```
python

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Kronecker product
kron = np.kron(A, B)

# Vectorization (column-wise flattening)
vec_A = A.flatten('F') # Fortran-style (column-major)

# Property:  $\text{vec}(ABC) = (C^T \otimes A)\text{vec}(B)$ 
```

ML Applications:

- Convolutional neural networks (CNN operations)
 - Tensor operations
 - Graph neural networks
-

10. ADVANCED CALCULUS & OPTIMIZATION

Hessian Matrix

Definition: Matrix of second-order partial derivatives

```
python
```

```

import sympy as sp

x, y = sp.symbols('x y')
f = x**3 + y**3 - 3*x*y

# Hessian matrix:  $H[i,j] = \partial^2 f / \partial x_i \partial x_j$ 
hessian = sp.hessian(f, (x, y))
#  $\begin{bmatrix} 6x & -3 \\ -3 & 6y \end{bmatrix}$ 

# Numerical Hessian
def numerical_hessian(f, x, eps=1e-5):
    n = len(x)
    H = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            x_pp = x.copy()
            x_pp[i] += eps
            x_pp[j] += eps

            x_pm = x.copy()
            x_pm[i] += eps
            x_pm[j] -= eps

            x_mp = x.copy()
            x_mp[i] -= eps
            x_mp[j] += eps

            x_mm = x.copy()
            x_mm[i] -= eps
            x_mm[j] -= eps

            H[i,j] = (f(x_pp) - f(x_pm) - f(x_mp) + f(x_mm)) / (4 * eps**2)
    return H

```

ML Applications:

- Newton's method optimization
- Determining local minima/maxima (convexity analysis)
- Second-order optimization methods (L-BFGS)

Lagrange Multipliers

Constrained Optimization: Optimize $f(x)$ subject to $g(x) = 0$

$\nabla f = \lambda \nabla g$ (at optimal point)

python

*# Example: Maximize $f(x,y) = x*y$ subject to $x^2 + y^2 = 1$*

```
def lagrangian(x, y, lambda_):  
    return x*y - lambda_ * (x**2 + y**2 - 1)
```

Solve: $\nabla L = 0$

$y - 2\lambda x = 0$

$x - 2\lambda y = 0$

$x^2 + y^2 - 1 = 0$

Solution: $x = y = \pm 1/\sqrt{2}$, $\lambda = \pm 1/2$

ML Applications:

- Support Vector Machines (SVM)
- Constrained neural network training
- Dual formulations

Taylor Series Approximation

python

$f(x) \approx f(a) + f'(a)(x-a) + f''(a)(x-a)^2/2! + \dots$

```
def taylor_approximation(f, df, ddf, a, x, order=2):  
    """Taylor series approximation around point a"""  
    result = f(a)  
    if order >= 1:  
        result += df(a) * (x - a)  
    if order >= 2:  
        result += 0.5 * ddf(a) * (x - a)**2  
    return result
```

Used in Newton's method:

$x_{new} = x - f'(x) / f''(x)$

ML Applications:

- Function approximation
- Understanding gradient descent convergence
- Neural network analysis

Jacobian Matrix

Definition: Matrix of first-order partial derivatives for vector functions

```
python

# For  $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ , Jacobian  $J[i,j] = \partial f_i / \partial x_j$ 

def jacobian(func, x, eps=1e-7):
    """Numerical Jacobian computation"""
    x = np.array(x, dtype=float)
    f0 = func(x)
    n = len(x)
    m = len(f0) if hasattr(f0, '__len__') else 1
    J = np.zeros((m, n))

    for j in range(n):
        x_plus = x.copy()
        x_plus[j] += eps
        f_plus = func(x_plus)
        J[:, j] = (f_plus - f0) / eps

    return J

# Example: Neural network layer
def layer(x):
    W = np.array([[1, 2], [3, 4], [5, 6]])
    b = np.array([1, 1, 1])
    return W @ x + b

x = np.array([1.0, 2.0])
J = jacobian(layer, x)
# J equals W (the weight matrix)
```

ML Applications:

- Backpropagation in neural networks
 - Sensitivity analysis
 - Coordinate transformations
-

11. PROBABILITY THEORY DEEP DIVE

Conditional Probability & Independence

```
python

# Joint probability
def joint_prob(p_a, p_b_given_a):
    return p_a * p_b_given_a

# Marginal probability
def marginal_prob(joint_probs):
    return np.sum(joint_probs, axis=1)

# Independence test:  $P(A \cap B) = P(A) \cdot P(B)$ 
def is_independent(p_a, p_b, p_ab):
    return np.abs(p_ab - p_a * p_b) < 1e-10
```

Maximum Likelihood Estimation (MLE)

```
python

# Example: Estimating mean and variance of Gaussian

def gaussian_mle(data):
    """MLE for Gaussian parameters"""
    mu_hat = np.mean(data)
    sigma_hat = np.std(data, ddof=0) # MLE uses N, not N-1
    return mu_hat, sigma_hat

# Log-likelihood for Gaussian
def log_likelihood_gaussian(data, mu, sigma):
    n = len(data)
    ll = -n/2 * np.log(2 * np.pi * sigma**2)
    ll -= np.sum((data - mu)**2) / (2 * sigma**2)
    return ll

# Example data
data = np.random.normal(5, 2, 1000)
mu_mle, sigma_mle = gaussian_mle(data)
```

ML Applications:

- Parameter estimation in probabilistic models
- Training generative models
- Logistic regression

Maximum A Posteriori (MAP) Estimation

Combines prior knowledge with data

$$\theta_{\text{MAP}} = \operatorname{argmax} P(\theta|\mathbf{D}) = \operatorname{argmax} P(\mathbf{D}|\theta) \cdot P(\theta)$$

```
python

def map_estimation(data, prior_mean, prior_var, likelihood_var):
    """MAP estimation for Gaussian with Gaussian prior"""
    n = len(data)
    data_mean = np.mean(data)

    # Posterior mean (MAP estimate)
    posterior_precision = 1/prior_var + n/likelihood_var
    posterior_mean = (prior_mean/prior_var + n*data_mean/likelihood_var) / posterior_precision

    return posterior_mean

# Example: Prior belief  $\mu=0$ , observe data with mean 5
data = np.random.normal(5, 1, 100)
map_estimate = map_estimation(data, prior_mean=0, prior_var=10, likelihood_var=1)
# MAP estimate is between 0 and 5, weighted by certainties
```

ML Applications:

- Bayesian neural networks
- Regularization (L2 = Gaussian prior)
- Hyperparameter tuning

Covariance & Correlation

Covariance: $Cov(X, Y) = E[(X - \mu_x)(Y - \mu_y)]$

```
def covariance(x, y):  
    return np.mean((x - np.mean(x)) * (y - np.mean(y)))
```

Covariance matrix

```
def covariance_matrix(X):  
    """X is n_samples × n_features"""  
    X_centered = X - np.mean(X, axis=0)  
    return (X_centered.T @ X_centered) / len(X)
```

Correlation: $\rho(X, Y) = Cov(X, Y) / (\sigma_x \cdot \sigma_y)$

```
def correlation(x, y):  
    return covariance(x, y) / (np.std(x) * np.std(y))
```

Using numpy

```
X = np.random.randn(100, 3)  
cov_matrix = np.cov(X.T)  
corr_matrix = np.corrcoef(X.T)
```

ML Applications:

- Feature correlation analysis
- PCA computation
- Portfolio optimization

Multivariate Gaussian Distribution

python

```

from scipy.stats import multivariate_normal

# PDF:  $f(x) = (1/\sqrt{(2\pi)^k |\Sigma|}) \exp(-\frac{1}{2}(x-\mu)^T \Sigma^{-1} (x-\mu))$ 

mu = np.array([0, 0])
cov = np.array([[1, 0.5], [0.5, 1]])

# Sample from multivariate Gaussian
samples = np.random.multivariate_normal(mu, cov, 1000)

# Evaluate PDF
mvn = multivariate_normal(mean=mu, cov=cov)
pdf_value = mvn.pdf([0, 0])

# Mahalanobis distance:  $d = \sqrt{(x-\mu)^T \Sigma^{-1} (x-\mu)}$ 
def mahalanobis_distance(x, mu, cov_inv):
    diff = x - mu
    return np.sqrt(diff.T @ cov_inv @ diff)

```

ML Applications:

- Gaussian Mixture Models (GMM)
- Anomaly detection
- Multivariate regression

12. INFORMATION THEORY APPLICATIONS

Mutual Information

Measures dependency between variables

$$I(X;Y) = H(X) + H(Y) - H(X,Y)$$

python

```

def mutual_information(x, y, bins=10):
    """Estimate mutual information"""
    # Create 2D histogram
    hist_2d, _, _ = np.histogram2d(x, y, bins=bins)
    hist_2d = hist_2d / np.sum(hist_2d) # Normalize

    # Marginal distributions
    p_x = np.sum(hist_2d, axis=1)
    p_y = np.sum(hist_2d, axis=0)

    # Mutual information
    mi = 0
    for i in range(bins):
        for j in range(bins):
            if hist_2d[i, j] > 0:
                mi += hist_2d[i, j] * np.log(hist_2d[i, j] / (p_x[i] * p_y[j]))

    return mi

# Example
x = np.random.randn(1000)
y = x + np.random.randn(1000) * 0.5 # Correlated
mi = mutual_information(x, y)

```

ML Applications:

- Feature selection
- Image registration
- Information bottleneck theory in deep learning

Jensen-Shannon Divergence

Symmetric version of KL divergence

$JSD(P||Q) = \frac{1}{2}D_{KL}(P||M) + \frac{1}{2}D_{KL}(Q||M)$ where $M = \frac{1}{2}(P + Q)$

python

```
def js_divergence(p, q):
    """Jensen-Shannon divergence"""
    p = np.array(p)
    q = np.array(q)
    m = 0.5 * (p + q)

    return 0.5 * kl_divergence(p, m) + 0.5 * kl_divergence(q, m)

# Properties:  $0 \leq JSD \leq 1$ , symmetric
```

ML Applications:

- GANs (Generative Adversarial Networks)
- Model comparison
- Clustering evaluation

Perplexity

Measures how well a probability model predicts a sample

Perplexity = $2^{H(p)}$ = $\exp(H(p))$

```
python

def perplexity(probabilities):
    """Lower is better"""
    h = entropy(probabilities)
    return 2 ** h

# For language models
def language_model_perplexity(model_probs, actual_sequence):
    """
    model_probs: predicted probability for each word
    Lower perplexity = better model
    """
    log_probs = np.log2(model_probs)
    avg_log_prob = np.mean(log_probs)
    return 2 ** (-avg_log_prob)
```

ML Applications:

- Language model evaluation
- Topic modeling
- t-SNE visualization

13. ADVANCED OPTIMIZATION TECHNIQUES

Momentum

Accelerates gradient descent in relevant direction

```
python

def sgd_with_momentum(grad_fn, w, lr=0.01, momentum=0.9, iterations=1000):
    v = np.zeros_like(w) # Velocity

    for t in range(iterations):
        grad = grad_fn(w)
        v = momentum * v - lr * grad
        w = w + v

    return w

# Physical intuition: ball rolling down hill
#  $v_t = \beta v_{t-1} - \alpha \nabla L$ 
#  $w_t = w_{t-1} + v_t$ 
```

Nesterov Accelerated Gradient

```
python

def nesterov_momentum(grad_fn, w, lr=0.01, momentum=0.9, iterations=1000):
    v = np.zeros_like(w)

    for t in range(iterations):
        # Look ahead
        w_ahead = w + momentum * v
        grad = grad_fn(w_ahead)
        v = momentum * v - lr * grad
        w = w + v

    return w
```

RMSprop

Adapts learning rate for each parameter

```
python
```

```
def rmsprop(grad_fn, w, lr=0.001, decay=0.9, eps=1e-8, iterations=1000):  
    cache = np.zeros_like(w)  
  
    for t in range(iterations):  
        grad = grad_fn(w)  
        cache = decay * cache + (1 - decay) * grad**2  
        w = w - lr * grad / (np.sqrt(cache) + eps)  
  
    return w
```

AdaGrad

```
python  
  
def adagrad(grad_fn, w, lr=0.01, eps=1e-8, iterations=1000):  
    cache = np.zeros_like(w)  
  
    for t in range(iterations):  
        grad = grad_fn(w)  
        cache += grad**2  
        w = w - lr * grad / (np.sqrt(cache) + eps)  
  
    return w
```

Learning Rate Schedules

```
python
```

```

# Step decay
def step_decay(epoch, initial_lr=0.1, drop=0.5, epochs_drop=10):
    return initial_lr * (drop ** np.floor(epoch / epochs_drop))

# Exponential decay
def exponential_decay(epoch, initial_lr=0.1, k=0.1):
    return initial_lr * np.exp(-k * epoch)

# Cosine annealing
def cosine_annealing(epoch, initial_lr=0.1, T_max=100):
    return initial_lr * (1 + np.cos(np.pi * epoch / T_max)) / 2

# Warm restart
def cosine_annealing_warm_restart(epoch, initial_lr=0.1, T_0=10, T_mult=2):
    T_cur = epoch
    T_i = T_0
    while T_cur >= T_i:
        T_cur -= T_i
        T_i *= T_mult
    return initial_lr * (1 + np.cos(np.pi * T_cur / T_i)) / 2

```

ML Applications: Training stability, faster convergence, escaping local minima

14. DIMENSIONALITY REDUCTION TECHNIQUES

t-SNE (t-Distributed Stochastic Neighbor Embedding)

```

python

from sklearn.manifold import TSNE

# Non-linear dimensionality reduction for visualization
tsne = TSNE(n_components=2, perplexity=30, n_iter=1000)
X_embedded = tsne.fit_transform(X)

# Key concepts:
# - Preserves local structure
# - Converts distances to probabilities
# - Minimizes KL divergence between high and low dimensional distributions

```

Autoencoders

```

python

```

Neural network for dimensionality reduction

class Autoencoder:

def __init__(self, input_dim, encoding_dim):

self.input_dim = input_dim

self.encoding_dim = encoding_dim

def encode(self, x, W_enc, b_enc):

return sigmoid(x @ W_enc + b_enc)

def decode(self, h, W_dec, b_dec):

return sigmoid(h @ W_dec + b_dec)

def reconstruct(self, x, W_enc, b_enc, W_dec, b_dec):

h = self.encode(x, W_enc, b_enc)

return self.decode(h, W_dec, b_dec)

Loss: reconstruction error

$L = ||x - \text{decoder}(\text{encoder}(x))||^2$

Linear Discriminant Analysis (LDA)

python

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

Supervised dimensionality reduction

Maximizes class separability

lda = LinearDiscriminantAnalysis(n_components=2)

X_lda = lda.fit_transform(X, y)

Maximizes: (between-class variance) / (within-class variance)

Independent Component Analysis (ICA)

python

from sklearn.decomposition import FastICA

Separates multivariate signal into independent sources

ica = FastICA(n_components=3)

S = ica.fit_transform(X) *# Sources*

A = ica.mixing_ *# Mixing matrix*

Applications: Blind source separation, feature extraction

15. KERNEL METHODS

Kernel Trick

Implicit mapping to high-dimensional space

```
python

# Linear kernel:  $K(x, y) = x^T y$ 
def linear_kernel(x, y):
    return np.dot(x, y)

# Polynomial kernel:  $K(x, y) = (x^T y + c)^d$ 
def polynomial_kernel(x, y, degree=3, c=1):
    return (np.dot(x, y) + c) ** degree

# RBF/Gaussian kernel:  $K(x, y) = \exp(-\gamma ||x - y||^2)$ 
def rbf_kernel(x, y, gamma=0.1):
    return np.exp(-gamma * np.linalg.norm(x - y)**2)

# Sigmoid kernel:  $K(x, y) = \tanh(\alpha x^T y + c)$ 
def sigmoid_kernel(x, y, alpha=0.1, c=1):
    return np.tanh(alpha * np.dot(x, y) + c)
```

Kernel Matrix (Gram Matrix)

```
python

def compute_kernel_matrix(X, kernel_fn):
    """Compute pairwise kernel values"""
    n = len(X)
    K = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            K[i, j] = kernel_fn(X[i], X[j])
    return K

# Properties of valid kernels (Mercer's theorem):
# 1. Symmetric:  $K(x, y) = K(y, x)$ 
# 2. Positive semi-definite
```

ML Applications:

- Support Vector Machines
- Kernel PCA

- Gaussian Processes
-

16. SAMPLING & MONTE CARLO METHODS

Monte Carlo Integration

python

```
def monte_carlo_integration(f, a, b, n_samples=10000):  
    """Estimate integral of f from a to b"""  
    samples = np.random.uniform(a, b, n_samples)  
    estimate = (b - a) * np.mean(f(samples))  
    return estimate
```

Example: Estimate π

```
def estimate_pi(n_samples=100000):  
    x = np.random.uniform(-1, 1, n_samples)  
    y = np.random.uniform(-1, 1, n_samples)  
    inside = (x**2 + y**2) <= 1  
    return 4 * np.sum(inside) / n_samples
```

Markov Chain Monte Carlo (MCMC)

python

```

def metropolis_hastings(target_pdf, proposal_std=0.5, n_samples=10000):
    """MCMC sampling from arbitrary distribution"""
    samples = [0] # Initial state

    for _ in range(n_samples - 1):
        current = samples[-1]

        # Propose new state
        proposed = current + np.random.normal(0, proposal_std)

        # Acceptance probability
        acceptance = min(1, target_pdf(proposed) / target_pdf(current))

        if np.random.rand() < acceptance:
            samples.append(proposed)
        else:
            samples.append(current)

    return np.array(samples)

# Example: Sample from Gaussian
target = lambda x: np.exp(-0.5 * x**2)
samples = metropolis_hastings(target)

```

Gibbs Sampling

python

```
def gibbs_sampling(n_samples=1000):  
    """Sample from bivariate Gaussian"""  
    x = 0  
    y = 0  
    samples = []  
  
    rho = 0.8 # Correlation  
  
    for _ in range(n_samples):  
        # Sample  $x | y$   
        x = np.random.normal(rho * y, np.sqrt(1 - rho**2))  
  
        # Sample  $y | x$   
        y = np.random.normal(rho * x, np.sqrt(1 - rho**2))  
  
        samples.append([x, y])  
  
    return np.array(samples)
```

ML Applications:

- Bayesian inference
 - Training generative models
 - Uncertainty quantification
-

17. TIME SERIES & SEQUENTIAL DATA

Autocorrelation

python

```

def autocorrelation(x, lag):
    """Correlation of signal with itself at different lags"""
    n = len(x)
    x_mean = np.mean(x)
    c0 = np.sum((x - x_mean)**2) / n

    if lag == 0:
        return 1.0

    c_lag = np.sum((x[:-lag] - x_mean) * (x[lag:] - x_mean)) / n
    return c_lag / c0

# Compute for multiple lags
def acf(x, max_lag=20):
    return [autocorrelation(x, lag) for lag in range(max_lag + 1)]

```

Moving Average & Exponential Smoothing

```

python

def moving_average(x, window=5):
    """Simple moving average"""
    return np.convolve(x, np.ones(window)/window, mode='valid')

def exponential_smoothing(x, alpha=0.3):
    """Exponentially weighted moving average"""
    result = [x[0]]
    for i in range(1, len(x)):
        result.append(alpha * x[i] + (1 - alpha) * result[-1])
    return np.array(result)

# Double exponential smoothing (Holt's method)
def double_exponential_smoothing(x, alpha=0.3, beta=0.1):
    level = [x[0]]
    trend = [x[1] - x[0]]
    result = [x[0]]

    for i in range(1, len(x)):
        level.append(alpha * x[i] + (1 - alpha) * (level[-1] + trend[-1]))
        trend.append(beta * (level[-1] - level[-2]) + (1 - beta) * trend[-1])
        result.append(level[-1] + trend[-1])

    return np.array(result)

```

Recurrent Neural Network Math

python

RNN forward pass

```
def rnn_forward(x_sequence, W_xh, W_hh, W_hy, b_h, b_y):
```

```
    """
```

```
    x_sequence: (time_steps, input_dim)
```

```
    h_t = tanh(W_xh @ x_t + W_hh @ h_{t-1} + b_h)
```

```
    y_t = W_hy @ h_t + b_y
```

```
    """
```

```
    h = np.zeros(W_hh.shape[0])
```

```
    outputs = []
```

```
    for x_t in x_sequence:
```

```
        h = np.tanh(W_xh @ x_t + W_hh @ h + b_h)
```

```
        y = W_hy @ h + b_y
```

```
        outputs.append(y)
```

```
    return np.array(outputs), h
```

LSTM cell

```
def lstm_cell(x_t, h_prev, c_prev, W_f, W_i, W_o, W_c, b_f, b_i, b_o, b_c):
```

```
    """Long Short-Term Memory cell"""
```

Forget gate

```
f_t = sigmoid(W_f @ np.concatenate([h_prev, x_t]) + b_f)
```

Input gate

```
i_t = sigmoid(W_i @ np.concatenate([h_prev, x_t]) + b_i)
```

Cell update

```
c_tilde = np.tanh(W_c @ np.concatenate([h_prev, x_t]) + b_c)
```

```
c_t = f_t * c_prev + i_t * c_tilde
```

Output gate

```
o_t = sigmoid(W_o @ np.concatenate([h_prev, x_t]) + b_o)
```

```
h_t = o_t * np.tanh(c_t)
```

```
return h_t, c_t
```

ML Applications: Natural language processing, stock prediction, speech recognition

18. GRAPH THEORY FOR ML

Graph Representations

python

Adjacency Matrix

```
def create_adjacency_matrix(edges, n_nodes):
```

```
    A = np.zeros((n_nodes, n_nodes))
```

```
    for i, j in edges:
```

```
        A[i, j] = 1
```

```
        A[j, i] = 1 # Undirected
```

```
    return A
```

Degree Matrix

```
def degree_matrix(A):
```

```
    return np.diag(np.sum(A, axis=1))
```

Laplacian Matrix: $L = D - A$

```
def laplacian_matrix(A):
```

```
    D = degree_matrix(A)
```

```
    return D - A
```

Normalized Laplacian: $L_{norm} = D^{-1/2} L D^{-1/2}$

```
def normalized_laplacian(A):
```

```
    D = degree_matrix(A)
```

```
    D_inv_sqrt = np.diag(1.0 / np.sqrt(np.diag(D)))
```

```
    L = laplacian_matrix(A)
```

```
    return D_inv_sqrt @ L @ D_inv_sqrt
```

Graph Convolutional Networks (GCN)

python

```
def gcn_layer(X, A, W):
    """
    X: Node features (n_nodes × feature_dim)
    A: Adjacency matrix (n_nodes × n_nodes)
    W: Weight matrix (feature_dim × output_dim)
    """
    # Add self-loops
    A_tilde = A + np.eye(A.shape[0])

    # Degree matrix
    D_tilde = degree_matrix(A_tilde)
    D_inv_sqrt = np.diag(1.0 / np.sqrt(np.diag(D_tilde)))

    # Normalized adjacency
    A_hat = D_inv_sqrt @ A_tilde @ D_inv_sqrt

    # Propagation:  $H = \sigma(A\_hat @ X @ W)$ 
    return relu(A_hat @ X @ W)
```

PageRank Algorithm

python

```

def pagerank(A, damping=0.85, max_iter=100, tol=1e-6):
    """
    Compute PageRank scores
     $PR(i) = (1-d)/N + d \cdot \sum (PR(j)/L(j))$  for all j linking to i
    """
    n = A.shape[0]

    # Normalize adjacency matrix (column stochastic)
    out_degree = np.sum(A, axis=0)
    out_degree[out_degree == 0] = 1 # Avoid division by zero
    M = A / out_degree

    # Initialize PageRank scores
    pr = np.ones(n) / n

    for _ in range(max_iter):
        pr_new = (1 - damping) / n + damping * M @ pr

        if np.linalg.norm(pr_new - pr) < tol:
            break

        pr = pr_new

    return pr

```

ML Applications: Node importance, recommendation systems, graph neural networks

19. NUMERICAL METHODS & STABILITY

Condition Number

Measures sensitivity of function output to input perturbations

```
python
```

```
def condition_number(A):
    """
     $\kappa(A) = \|A\| \cdot \|A^{-1}\|$ 
    High condition number = ill-conditioned (numerical instability)
    """
    return np.linalg.cond(A)

# Example
A_good = np.array([[1, 0], [0, 1]])
A_bad = np.array([[1, 1], [1, 1.0001]])

print(condition_number(A_good)) #  $\approx 1$  (well-conditioned)
print(condition_number(A_bad)) #  $\gg 1$  (ill-conditioned)
```

ML Applications: Training stability, gradient checking, matrix inversions

Numerical Gradient Checking

```
python

def gradient_check(f, grad_f, x, eps=1e-5, tolerance=1e-7):
    """
    Verify gradient implementation
    Numerical:  $(f(x+\epsilon) - f(x-\epsilon)) / (2\epsilon)$ 
    """
    analytical_grad = grad_f(x)
    numerical_grad = np.zeros_like(x)

    for i in range(len(x)):
        x_plus = x.copy()
        x_plus[i] += eps
        x_minus = x.copy()
        x_minus[i] -= eps

        numerical_grad[i] = (f(x_plus) - f(x_minus)) / (2 * eps)

    # Relative error
    relative_error = np.linalg.norm(analytical_grad - numerical_grad) / \
        (np.linalg.norm(analytical_grad) + np.linalg.norm(numerical_grad))

    if relative_error < tolerance:
        print("✓ Gradient check passed!")
    else:
        print(f"✗ Gradient check failed! Relative error: {relative_error}")

    return relative_error
```

Numerical Integration Methods

python

Trapezoidal rule

```
def trapezoidal(f, a, b, n=1000):
```

```
    x = np.linspace(a, b, n)
```

```
    y = f(x)
```

```
    return (b - a) * (y[0] + 2*np.sum(y[1:-1]) + y[-1]) / (2*n)
```

Simpson's rule

```
def simpsons(f, a, b, n=1000):
```

```
    if n % 2 == 1:
```

```
        n += 1
```

```
    x = np.linspace(a, b, n+1)
```

```
    y = f(x)
```

```
    h = (b - a) / n
```

```
    return h/3 * (y[0] + 4*np.sum(y[1:-1:2]) + 2*np.sum(y[2:-1:2]) + y[-1])
```

20. FOURIER ANALYSIS

Discrete Fourier Transform (DFT)

python

Manual DFT implementation

```
def dft(x):  
    """  
     $X[k] = \sum x[n] \cdot e^{(-2\pi i k n / N)}$   
    Converts time domain to frequency domain  
    """  
    N = len(x)  
    X = np.zeros(N, dtype=complex)  
  
    for k in range(N):  
        for n in range(N):  
            X[k] += x[n] * np.exp(-2j * np.pi * k * n / N)  
  
    return X
```

Fast Fourier Transform ($O(N \log N)$)

```
def apply_fft(signal):  
    return np.fft.fft(signal)
```

```
def apply_ifft(spectrum):  
    return np.fft.ifft(spectrum)
```

Example: Detect frequency components

```
t = np.linspace(0, 1, 500)  
signal = np.sin(2 * np.pi * 5 * t) + 0.5 * np.sin(2 * np.pi * 10 * t)  
spectrum = np.fft.fft(signal)  
frequencies = np.fft.fftfreq(len(t), t[1] - t[0])
```

Convolution Theorem

Convolution in time $\leftrightarrow$ Multiplication in frequency

python

```
def convolve_fft(signal, kernel):
    """Fast convolution using FFT"""
    # Pad to avoid circular convolution
    n = len(signal) + len(kernel) - 1
    signal_padded = np.pad(signal, (0, n - len(signal)))
    kernel_padded = np.pad(kernel, (0, n - len(kernel)))

    # Convolution via FFT
    return np.fft.ifft(np.fft.fft(signal_padded) * np.fft.fft(kernel_padded)).real

# Standard convolution (slower)
def convolve_direct(signal, kernel):
    return np.convolve(signal, kernel, mode='full')
```

ML Applications:

- Convolutional Neural Networks
- Audio processing
- Image filtering

Wavelet Transform

```
python

import pywt

# Discrete Wavelet Transform
def dwt_decompose(signal, wavelet='db4', level=3):
    """Multi-level wavelet decomposition"""
    coeffs = pywt.wavedec(signal, wavelet, level=level)
    return coeffs

# Reconstruction
def dwt_reconstruct(coeffs, wavelet='db4'):
    return pywt.waverec(coeffs, wavelet)

# Example
signal = np.sin(2 * np.pi * np.linspace(0, 1, 1024))
coeffs = dwt_decompose(signal)
reconstructed = dwt_reconstruct(coeffs)
```

ML Applications: Feature extraction, denoising, compression

21. ATTENTION MECHANISMS

Scaled Dot-Product Attention

```
python

def scaled_dot_product_attention(Q, K, V, mask=None):
    """
    Attention(Q, K, V) = softmax(QK^T /  $\sqrt{d_k}$ )V

    Q: Query ( $n_{\text{queries}} \times d_k$ )
    K: Key ( $n_{\text{keys}} \times d_k$ )
    V: Value ( $n_{\text{keys}} \times d_v$ )
    """
    d_k = Q.shape[-1]

    # Compute attention scores
    scores = Q @ K.T / np.sqrt(d_k)

    # Apply mask if provided (for padding or causality)
    if mask is not None:
        scores = scores + mask * -1e9

    # Softmax to get attention weights
    attention_weights = softmax(scores)

    # Weighted sum of values
    output = attention_weights @ V

    return output, attention_weights

# Example usage
d_k, d_v = 64, 64
n_queries, n_keys = 10, 20

Q = np.random.randn(n_queries, d_k)
K = np.random.randn(n_keys, d_k)
V = np.random.randn(n_keys, d_v)

output, weights = scaled_dot_product_attention(Q, K, V)
```

Multi-Head Attention

```
python
```

```

def multi_head_attention(Q, K, V, num_heads=8):
    """
    Split into multiple heads, apply attention, concatenate
    """
    d_model = Q.shape[-1]
    d_k = d_model // num_heads

    # Split into heads
    Q_heads = Q.reshape(Q.shape[0], num_heads, d_k)
    K_heads = K.reshape(K.shape[0], num_heads, d_k)
    V_heads = V.reshape(V.shape[0], num_heads, d_k)

    # Apply attention to each head
    outputs = []
    for i in range(num_heads):
        out, _ = scaled_dot_product_attention(
            Q_heads[:, i, :],
            K_heads[:, i, :],
            V_heads[:, i, :]
        )
        outputs.append(out)

    # Concatenate heads
    concat = np.concatenate(outputs, axis=-1)

    return concat

```

Self-Attention

```

python

def self_attention(X, W_q, W_k, W_v):
    """
    Self-attention: Q = K = V = X
    Used in Transformers
    """
    Q = X @ W_q
    K = X @ W_k
    V = X @ W_v

    return scaled_dot_product_attention(Q, K, V)

```

ML Applications: Transformers, BERT, GPT, Vision Transformers

22. MATRIX CALCULUS

Vector-by-Vector Derivatives

```
python

#  $\partial(Ax)/\partial x = A^T$ 
def jacobian_linear(A):
    return A.T

#  $\partial(x^T A x)/\partial x = (A + A^T)x$ 
def gradient_quadratic(x, A):
    return (A + A.T) @ x

# If A is symmetric:  $\partial(x^T A x)/\partial x = 2Ax$ 
def gradient_quadratic_symmetric(x, A):
    return 2 * A @ x
```

Matrix-by-Scalar Derivatives

```
python

# Chain rule for matrices
#  $\partial L/\partial W = \partial L/\partial Y \cdot \partial Y/\partial W$ 

# Example: Linear layer
#  $Y = XW + b$ 
#  $\partial L/\partial W = X^T \cdot \partial L/\partial Y$ 
#  $\partial L/\partial b = \sum \partial L/\partial Y$  (sum over batch)
#  $\partial L/\partial X = \partial L/\partial Y \cdot W^T$ 

def linear_backward(dL_dY, X, W):
    """Backprop through linear layer"""
    dL_dW = X.T @ dL_dY
    dL_db = np.sum(dL_dY, axis=0)
    dL_dX = dL_dY @ W.T

    return dL_dX, dL_dW, dL_db
```

Common Derivatives in ML

```
python
```

```
# Softmax derivative
```

```
def softmax_derivative(s):
```

```
    """
```

```
    s: softmax output
```

```
    Returns Jacobian matrix
```

```
    """
```

```
    return np.diag(s) - np.outer(s, s)
```

```
# Batch normalization derivative
```

```
def batchnorm_backward(dout, x, gamma, eps=1e-5):
```

```
    """
```

```
    Backprop through batch normalization
```

```
    """
```

```
    N, D = x.shape
```

```
# Forward pass values (should be cached)
```

```
    mu = np.mean(x, axis=0)
```

```
    var = np.var(x, axis=0)
```

```
    std = np.sqrt(var + eps)
```

```
    x_norm = (x - mu) / std
```

```
# Gradients
```

```
    dgamma = np.sum(dout * x_norm, axis=0)
```

```
    dbeta = np.sum(dout, axis=0)
```

```
    dx_norm = dout * gamma
```

```
    dvar = np.sum(dx_norm * (x - mu) * -0.5 * (var + eps)**(-1.5), axis=0)
```

```
    dmu = np.sum(dx_norm * -1/std, axis=0) + dvar * np.sum(-2*(x - mu), axis=0) / N
```

```
    dx = dx_norm / std + dvar * 2 * (x - mu) / N + dmu / N
```

```
    return dx, dgamma, dbeta
```

23. ENSEMBLE METHODS MATHEMATICS

Bagging (Bootstrap Aggregating)

```
python
```

```

def bootstrap_sample(X, y, n_samples=None):
    """Sample with replacement"""
    if n_samples is None:
        n_samples = len(X)

    indices = np.random.choice(len(X), n_samples, replace=True)
    return X[indices], y[indices]

def bagging_predict(models, X):
    """Average predictions from multiple models"""
    predictions = np.array([model.predict(X) for model in models])
    return np.mean(predictions, axis=0)

# Variance reduction:  $Var(avg) = Var(model) / n\_models$ 

```

Boosting Weight Updates

```

python

# AdaBoost weight update
def adaboost_weights(y_true, y_pred, weights):
    """
    Update sample weights based on errors
    """
    # Error rate
    errors = (y_true != y_pred).astype(float)
    error_rate = np.sum(weights * errors) / np.sum(weights)

    # Model weight
    alpha = 0.5 * np.log((1 - error_rate) / (error_rate + 1e-10))

    # Update sample weights
    new_weights = weights * np.exp(-alpha * y_true * y_pred)
    new_weights /= np.sum(new_weights) # Normalize

    return new_weights, alpha

# Gradient Boosting residual
def gradient_boosting_residual(y_true, y_pred):
    """
    Negative gradient of loss function
    For MSE: residual = y_true - y_pred
    """
    return y_true - y_pred

```

Random Forest Information Gain

python

```
def gini_impurity(y):  
    """  
    Gini = 1 -  $\sum p_i^2$   
    Lower is better (more pure)  
    """  
    classes, counts = np.unique(y, return_counts=True)  
    probabilities = counts / len(y)  
    return 1 - np.sum(probabilities**2)  
  
def information_gain(y_parent, y_left, y_right):  
    """  
    IG = Gini(parent) - weighted_average(Gini(children))  
    """  
    n = len(y_parent)  
    n_left, n_right = len(y_left), len(y_right)  
  
    gini_parent = gini_impurity(y_parent)  
    gini_left = gini_impurity(y_left)  
    gini_right = gini_impurity(y_right)  
  
    return gini_parent - (n_left/n * gini_left + n_right/n * gini_right)
```

24. BAYESIAN METHODS

Conjugate Priors

python

```
# Beta-Bernoulli conjugate pair
```

```
def beta_bernoulli_posterior(alpha_prior, beta_prior, successes, failures):
```

```
    """
```

```
    Prior: Beta( $\alpha$ ,  $\beta$ )
```

```
    Likelihood: Bernoulli
```

```
    Posterior: Beta( $\alpha + \text{successes}$ ,  $\beta + \text{failures}$ )
```

```
    """
```

```
    alpha_post = alpha_prior + successes
```

```
    beta_post = beta_prior + failures
```

```
# Posterior mean (MAP estimate)
```

```
    posterior_mean = alpha_post / (alpha_post + beta_post)
```

```
    return alpha_post, beta_post, posterior_mean
```

```
# Gaussian-Gaussian conjugate pair
```

```
def gaussian_gaussian_posterior(prior_mu, prior_var, data, likelihood_var):
```

```
    """
```

```
    Prior: N( $\mu_0$ ,  $\sigma^2$ )
```

```
    Likelihood: N( $x \mid \mu$ ,  $\sigma^2$ )
```

```
    Posterior: N( $\mu_{\text{post}}$ ,  $\sigma_{\text{post}}^2$ )
```

```
    """
```

```
    n = len(data)
```

```
    data_mean = np.mean(data)
```

```
# Posterior parameters
```

```
    posterior_precision = 1/prior_var + n/likelihood_var
```

```
    posterior_var = 1 / posterior_precision
```

```
    posterior_mu = posterior_var * (prior_mu/prior_var + n*data_mean/likelihood_var)
```

```
    return posterior_mu, posterior_var
```

Variational Inference

```
python
```

```

def mean_field_vi(data, n_iterations=1000):
    """
    Approximate posterior with factorized distribution
    Minimize KL(q||p)
    """
    # Initialize variational parameters
    mu_q = 0
    sigma_q = 1

    for _ in range(n_iterations):
        # ELBO (Evidence Lower BOund)
        #  $L = E_q[\log p(x,z)] - E_q[\log q(z)]$ 

        # Update variational parameters (example for Gaussian)
        mu_q = np.mean(data) # Simplified update
        sigma_q = np.std(data)

    return mu_q, sigma_q

# ELBO computation
def elbo(data, mu_q, sigma_q, mu_prior, sigma_prior):
    """Evidence Lower Bound"""
    # Reconstruction term
    recon = -0.5 * np.sum((data - mu_q)**2) / sigma_q**2

    # KL divergence term
    kl = 0.5 * (sigma_q**2 / sigma_prior**2 +
               (mu_q - mu_prior)**2 / sigma_prior**2 -
               1 + np.log(sigma_prior**2 / sigma_q**2))

    return recon - kl

```

Gaussian Processes

python

```

def gaussian_process_regression(X_train, y_train, X_test, kernel_fn, noise=1e-8):
    """
    GP regression:  $f(x) \sim \text{GP}(m(x), k(x, x'))$ 
    Posterior:  $p(f^*|X, y, X^*) = N(\mu^*, \Sigma^*)$ 
    """
    # Compute kernel matrices
    K = compute_kernel_matrix(X_train, kernel_fn)
    K_star = np.array([[kernel_fn(x_test, x_train)
                        for x_train in X_train]
                       for x_test in X_test])
    K_star_star = compute_kernel_matrix(X_test, kernel_fn)

    # Add noise to diagonal
    K_noisy = K + noise * np.eye(len(K))

    # Posterior mean:  $\mu^* = K^* K^{(-1)} y$ 
    K_inv = np.linalg.inv(K_noisy)
    mu_star = K_star @ K_inv @ y_train

    # Posterior covariance:  $\Sigma^* = K^{**} - K^* K^{(-1)} K^{*T}$ 
    cov_star = K_star_star - K_star @ K_inv @ K_star.T

    # Posterior standard deviation
    std_star = np.sqrt(np.diag(cov_star))

    return mu_star, std_star

```

ML Applications: Hyperparameter optimization, Bayesian neural networks, uncertainty quantification

25. ADVANCED APPLICATIONS

Generative Adversarial Networks (GANs)

```
python
```

GAN loss functions

```
def discriminator_loss(real_output, fake_output):  
    """  
     $L_D = -E[\log D(x)] - E[\log(1 - D(G(z)))]$   
    Maximize probability of correct classification  
    """  
    real_loss = -np.mean(np.log(real_output + 1e-8))  
    fake_loss = -np.mean(np.log(1 - fake_output + 1e-8))  
    return real_loss + fake_loss
```

```
def generator_loss(fake_output):  
    """  
     $L_G = -E[\log D(G(z))]$   
    Maximize probability of fooling discriminator  
    """  
    return -np.mean(np.log(fake_output + 1e-8))
```

Wasserstein GAN (WGAN)

```
def wasserstein_discriminator_loss(real_output, fake_output):  
    """  
     $L_D = E[D(G(z))] - E[D(x)]$   
    Earth Mover's Distance  
    """  
    return np.mean(fake_output) - np.mean(real_output)
```

```
def wasserstein_generator_loss(fake_output):  
    """  
     $L_G = -E[D(G(z))]$   
    """  
    return -np.mean(fake_output)
```

Variational Autoencoders (VAE)

python

```

def vae_loss(x, x_recon, mu, log_var):
    """
    L = Reconstruction Loss + KL Divergence
    """
    # Reconstruction loss (binary cross-entropy or MSE)
    recon_loss = np.sum((x - x_recon)**2)

    # KL divergence:  $KL(q(z|x) || p(z))$ 
    # For  $q = N(\mu, \sigma^2)$  and  $p = N(0, 1)$ :
    #  $KL = -0.5 * \Sigma(1 + \log(\sigma^2) - \mu^2 - \sigma^2)$ 
    kl_loss = -0.5 * np.sum(1 + log_var - mu**2 - np.exp(log_var))

    return recon_loss + kl_loss

# Reparameterization trick
def reparameterize(mu, log_var):
    """
     $z = \mu + \sigma \cdot \epsilon$ , where  $\epsilon \sim N(0, 1)$ 
    Makes sampling differentiable
    """
    std = np.exp(0.5 * log_var)
    eps = np.random.randn(*mu.shape)
    return mu + std * eps

```

Reinforcement Learning Value Functions

python

Bellman equation: $V(s) = \max_a [R(s,a) + \gamma V(s')]$

```
def value_iteration(rewards, transitions, gamma=0.9, theta=1e-6):
```

```
    """
```

```
    Dynamic programming to find optimal value function
```

```
    """
```

```
    n_states = rewards.shape[0]
```

```
    V = np.zeros(n_states)
```

```
    while True:
```

```
        delta = 0
```

```
        for s in range(n_states):
```

```
            v = V[s]
```

```
            # Bellman update
```

```
            V[s] = np.max([rewards[s, a] + gamma * np.sum(transitions[s, a] * V)
```

```
                           for a in range(rewards.shape[1])])
```

```
            delta = max(delta, abs(v - V[s]))
```

```
    if delta < theta:
```

```
        break
```

```
    return V
```

```
# Q-learning update
```

```
def q_learning_update(Q, s, a, r, s_next, alpha=0.1, gamma=0.9):
```

```
    """
```

```
     $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$ 
```

```
    """
```

```
    target = r + gamma * np.max(Q[s_next])
```

```
    Q[s, a] = Q[s, a] + alpha * (target - Q[s, a])
```

```
    return Q
```

Metric Learning

python

```
# Triplet loss
```

```
def triplet_loss(anchor, positive, negative, margin=0.2):  
    """  
    L = max(||f(a) - f(p)||2 - ||f(a) - f(n)||2 + margin, 0)  
    Encourages: d(anchor, positive) < d(anchor, negative)  
    """  
    pos_dist = np.sum((anchor - positive)**2)  
    neg_dist = np.sum((anchor - negative)**2)  
    loss = np.maximum(pos_dist - neg_dist + margin, 0)  
    return loss
```

```
# Contrastive loss
```

```
def contrastive_loss(x1, x2, y, margin=1.0):  
    """  
    y=1 (similar): L = ||x1 - x2||2  
    y=0 (dissimilar): L = max(margin - ||x1 - x2||, 0)2  
    """  
    distance = np.linalg.norm(x1 - x2)  
    if y == 1:  
        return distance**2  
    else:  
        return max(margin - distance, 0)**2
```

26. COMPUTATIONAL COMPLEXITY

Big O Notation for ML Algorithms

Algorithm	Training	Prediction	Space
Linear Regression	$O(nd^2 + d^3)$	$O(d)$	$O(d)$
Logistic Regression	$O(ndi)$	$O(d)$	$O(d)$
k-NN	$O(1)$	$O(nd)$	$O(nd)$
Decision Tree	$O(n \cdot d \cdot \log n)$	$O(\log n)$	$O(n)$
Random Forest	$O(k \cdot n \cdot d \cdot \log n)$	$O(k \cdot \log n)$	$O(k \cdot n)$
SVM	$O(n^2d)$ to $O(n^3d)$	$O(sv \cdot d)$	$O(sv \cdot d)$
Neural Network	$O(w \cdot i)$	$O(w)$	$O(w)$
PCA	$O(nd^2 + d^3)$	$O(d^2)$	$O(d^2)$

Algorithm	Training	Prediction	Space
k-Means	$O(k \cdot n \cdot d \cdot i)$	$O(k \cdot d)$	$O(k \cdot d)$

n=samples, d=features, i=iterations, k=trees/clusters, w=weights, sv=support vectors

This comprehensive cheat sheet provides mathematical foundations with practical implementations for real-world machine learning applications.